

Sprint 11 Structured Notes: Browser APIs, DOM, and Element Selection

0. What Sprint 11 Is About

Sprint 11 is the point where JavaScript starts connecting directly to the browser page.

Before Sprint 11, most of your JavaScript was about values and logic:

```
const name = "Ada";
const score = 90;
console.log(name, score);
```

In Sprint 11, you start using JavaScript to access real HTML elements on the page:

```
const title = document.querySelector("h1");
console.log(title.textContent);
```

Simple meaning:

Sprint 11 teaches you how JavaScript sees and accesses the webpage.

Main goal:

Use browser-provided APIs to access the page and browser environment.

By the end of Sprint 11, you should understand:

- what an API is
- what a browser API is
- what a web API is
- what the DOM is
- what the DOM tree is
- what nodes and elements are
- parent, child, sibling, ancestor, and descendant relationships
- the difference between `document`, `window`, and `navigator`
- how to select one element with `getElementById()`
- how to select one element with `querySelector()`
- how to select many elements with `querySelectorAll()`
- what a `NodeList` is
- how to loop through selected elements

- how to use `textContent`
- what `null` means when selecting elements
- the difference between one selected element and many selected elements
- common selector mistakes
- why script placement matters

Part A: Current Progress Before Sprint 11

1. What You Already Know

Sprint 11 depends on the first 10 sprints.

Earlier Sprint	What You Learned	How It Helps in Sprint 11
Sprint 1	JavaScript can respond to a webpage.	Sprint 11 uses JavaScript to access page elements.
Sprint 2	Values, variables, <code>let</code> , <code>const</code> , and types.	Selected HTML elements are stored in variables.
Sprint 3	Strings and text processing.	DOM elements often contain text that you read with <code>textContent</code> .
Sprint 4	Conditions and booleans.	You check whether an element exists before using it.
Sprint 5	Functions, callbacks, and scope.	Browser events and loops often use callback functions.
Sprint 6	Arrays and collections.	<code>querySelectorAll()</code> returns a collection-like value.
Sprint 7	Loops and iteration.	You loop through many selected elements.
Sprint 8	Objects and property access.	Browser APIs expose objects like <code>document</code> , <code>window</code> , and <code>navigator</code> .
Sprint 9	<code>forEach()</code> and callbacks.	You can use <code>forEach()</code> on a <code>NodeList</code> from <code>querySelectorAll()</code> .
Sprint 10	Sets and Maps as collection tools.	You now understand that JavaScript has different collection-like structures.

Simple summary:

Up to Sprint 10, you learned how JavaScript stores, organizes, and processes data. Sprint 11 teaches you how JavaScript connects that knowledge to the actual browser page.

2. The Main Shift in Sprint 11

Before Sprint 11, you mostly worked with JavaScript values:

```
const user = {  
  name: "Sangeeth",  
  active: true  
};  
  
console.log(user.name);
```

Now you start working with HTML elements:

```
<h1 id="title">Student Dashboard</h1>
```

```
const title = document.getElementById("title");  
console.log(title.textContent);
```

The important shift:

JavaScript is no longer only working with data inside your code. It is now accessing objects created by the browser from your HTML.

Part B: APIs

3. What Is an API?

API means:

Application Programming Interface

That sounds complex, but the beginner meaning is simple:

An API is a set of tools that one piece of software gives another piece of software.

Another simple way to say it:

An API is a menu of commands you are allowed to use.

For example, the browser gives JavaScript tools like:

```
document.querySelector();  
document.getElementById();  
window.innerWidth;  
navigator.language;
```

These are not random JavaScript names. They are tools provided by the browser.

4. API Mental Model

Imagine a restaurant menu.

You cannot walk into the kitchen and do anything you want. Instead, the menu tells you what you are allowed to order.

An API works in a similar way.

The browser says:

Here are the commands JavaScript is allowed to use to interact with the page and browser.

For example:

```
document.querySelector("h1");
```

This asks the browser:

Find the first `<h1>` element in the document.

5. Browser APIs

A browser API is an API built into the browser.

Examples:

```
document.querySelector("h1");  
window.innerWidth;  
navigator.onLine;
```

Browser APIs let JavaScript work with things like:

- the HTML page
- the browser window
- browser/device information
- events
- timers
- network requests
- storage
- location features

In Sprint 11, the main browser APIs are:

```
document  
window  
navigator
```

6. Web APIs

A web API is an API used in web applications.

Some web APIs are built into the browser. Some APIs can come from external services.

Beginner distinction:

API Type	Simple Meaning	Example
Browser API	Built into the browser	<code>document.querySelector()</code>
Web API	API used by web apps	DOM API, Fetch API, Storage API
Third-party API	API from an outside service	Weather data API, payment API, map API

Sprint 11 focuses on browser-provided APIs, especially the DOM API.

Part C: The DOM

7. What Is the DOM?

DOM means:

Document Object Model

Simple meaning:

The DOM is the browser's JavaScript-friendly version of your HTML page.

When you write HTML, the browser reads it and creates a structured model of the page.

HTML:

```
<h1>Hello</h1>
<p>Welcome</p>
<button>Click me</button>
```

The browser turns that into a tree-like structure that JavaScript can access.

Mental model:

```
document
├── html
│   └── body
│       ├── h1
│       ├── p
│       └── button
```

This structure is called the DOM tree.

8. Why the DOM Matters

JavaScript cannot directly work with raw HTML text as if it were normal code.

Instead, the browser converts the HTML into DOM objects.

Then JavaScript can find and use those objects.

Example:

```
<h1 id="mainTitle">Recipe</h1>
```

JavaScript can access that element:

```
const title = document.getElementById("mainTitle");
console.log(title.textContent);
```

Expected output:

Recipe

Simple meaning:

The DOM is the bridge between HTML and JavaScript.

9. DOM as a Tree

The DOM is usually described as a tree.

Example HTML:

```
<body>
  <section>
    <h2>Fruits</h2>
    <ul>
      <li>Apple</li>
      <li>Banana</li>
    </ul>
  </section>
</body>
```

DOM tree mental model:

```
body
├── section
│   ├── h2
│   └── ul
│       ├── li
│       └── li
```

A tree has pieces connected to other pieces.

That is why we use relationship words like:

- parent

- child
 - sibling
 - ancestor
 - descendant
-

Part D: Nodes and Elements

10. What Is a Node?

A node is one piece of the DOM tree.

Example:

```
<h1>Hello</h1>
<p>Welcome</p>
<button>Click me</button>
```

Each of these can become a node in the DOM:

- `<h1>Hello</h1>`
- `<p>Welcome</p>`
- `<button>Click me</button>`

Simple meaning:

A node is a single piece of the page structure.

11. What Is an Element?

An element is a specific kind of node that represents an HTML tag.

Examples of elements:

```
<h1>Hello</h1>
<p>Welcome</p>
<button>Click me</button>
<ul></ul>
<li></li>
<section></section>
```

Beginner meaning:

An element is an HTML thing JavaScript can find and use.

For most beginner DOM work, you will mostly deal with elements.

12. Node vs Element

Word	Meaning	Beginner Example
Node	Any piece of the DOM tree	element node, text node, comment node
Element	A node representing an HTML tag	<code><h1></code> , <code><p></code> , <code><button></code>

Beginner rule:

Every element is a node, but not every node is an element.

You do not need to master every node type yet. For Sprint 11, focus on selecting HTML elements.

Part E: DOM Relationships

13. Parent

A parent is the element directly above another element.

Example:

```
<ul>
  <li>Apple</li>
</ul>
```

Here:

```
ul
└─ li
```

The `<ul>` is the parent of the `<li>`.

Simple meaning:

A parent directly contains another element.

14. Child

A child is the element directly inside another element.

Example:

```
<ul>
  <li>Apple</li>
</ul>
```

The `<li>` is the child of the `<ul>`.

Simple meaning:

A child is directly inside a parent.

15. Sibling

Siblings are elements that share the same parent.

Example:

```
<ul>
  <li>Apple</li>
  <li>Banana</li>
  <li>Cherry</li>
</ul>
```

The three `<li>` elements are siblings because they all have the same parent: `<ul>`.

Simple meaning:

Siblings are elements at the same level under the same parent.

16. Ancestor

An ancestor is any element above another element in the DOM tree.

Example:

```
<body>
  <section>
    <ul>
      <li>Apple</li>
    </ul>
  </section>
</body>
```

For the `<li>` element, these are ancestors:

- `<ul>`
- `<section>`
- `<body>`
- `<html>`
- `document`

Simple meaning:

An ancestor is a parent, grandparent, great-grandparent, or anything above.

17. Descendant

A descendant is any element inside another element, at any depth.

Example:

```
<section>
  <ul>
    <li>Apple</li>
  </ul>
</section>
```

For the `<section>` element, these are descendants:

- `<ul>`
- `<li>`

Simple meaning:

A descendant is a child, grandchild, great-grandchild, or anything deeper inside.

18. Relationship Summary

Relationship	Simple Meaning	Example
Parent	Directly contains another element	<code>ul</code> is parent of <code>li</code>
Child	Directly inside another element	<code>li</code> is child of <code>ul</code>
Sibling	Same parent	multiple <code>li</code> elements inside one <code>ul</code>
Ancestor	Any element above	<code>body</code> is ancestor of <code>li</code>
Descendant	Any element inside	<code>li</code> is descendant of <code>body</code>

Part F: The Three Big Browser Objects

Sprint 11 introduces three important browser objects:

```
document
window
navigator
```

These are objects provided by the browser environment.

19. `document`

`document` represents the webpage.

Use `document` when you want to:

- find an element
- read page information
- access the DOM tree
- later change page content

Examples:

```
console.log(document.title);
```

```
const heading = document.querySelector("h1");
```

```
const button = document.getElementById("runBtn");
```

Simple meaning:

`document` is JavaScript's access point to the HTML page.

20. `window`

`window` represents the browser window or browser tab.

Use `window` when you want information about the browser window.

Examples:

```
console.log(window.innerWidth);  
console.log(window.innerHeight);
```

These can tell you the visible size of the browser window.

Simple meaning:

`window` represents the current browser window or tab.

21. `window` and Global Browser Features

Many browser features are available through `window`.

Example:

```
window.console.log("Hello");
```

Usually, you write:

```
console.log("Hello");
```

Both work because `console` is available globally in the browser environment.

Beginner note:

You do not always write `window.` explicitly, but many browser globals belong to `window`.

22. `navigator`

`navigator` gives information about the browser or device environment.

Examples:

```
console.log(navigator.language);  
console.log(navigator.onLine);
```

Possible output:

```
en-US  
true
```

Simple meaning:

`navigator` gives browser/device information, not page content.

Examples of information `navigator` may provide:

- language
- online/offline status
- browser environment details
- device capability information

23. `document` vs `window` vs `navigator`

Object	Represents	Use It For
<code>document</code>	The webpage	Finding and working with HTML elements
<code>window</code>	The browser window/tab	Window size and global browser features
<code>navigator</code>	Browser/device environment	Language, online status, device/browser info

Beginner memory:

```
document -> page  
window   -> browser window  
navigator -> browser/device info
```

Part G: Selecting Elements

24. What Does Selecting an Element Mean?

Selecting an element means:

Finding an HTML element and storing a reference to it in JavaScript.

Example HTML:

```
<h1 id="title">Hello</h1>
```

Example JavaScript:

```
const title = document.getElementById("title");
```

Now the variable `title` points to the `<h1>` element.

Simple meaning:

JavaScript cannot work with a page element until it finds that element first.

25. What Is a Reference?

A reference is a JavaScript value that points to something.

When you write:

```
const title = document.getElementById("title");
```

The variable `title` does not contain the text `"Hello"` by itself.

It contains a reference to this DOM element:

```
<h1 id="title">Hello</h1>
```

That reference lets you access properties of the element:

```
console.log(title.textContent);
```

Expected output:

```
Hello
```

Part H: `getElementById()`

26. What Is `getElementById()`?

`getElementById()` finds one element by its `id`.

Syntax:

```
document.getElementById("idName");
```

Example HTML:

```
<h1 id="mainTitle">JavaScript Practice</h1>
```

Example JavaScript:

```
const heading = document.getElementById("mainTitle");  
console.log(heading.textContent);
```

Expected output:

```
JavaScript Practice
```

Simple meaning:

Search the document for the element whose ID is `mainTitle`.

27. The `id` Attribute

An `id` gives an HTML element a unique name.

Example:

```
<button id="runBtn">Run</button>
```

The button's ID is:

```
runBtn
```

JavaScript can use that ID to find the button:

```
const button = document.getElementById("runBtn");
```

Important rule:

IDs should be unique on the page.

This means you should not use the same ID for multiple elements.

28. `getElementById()` Does Not Use `#`

This is correct:

```
document.getElementById("mainTitle");
```

This is wrong:

```
document.getElementById("#mainTitle");
```

Why?

Because `getElementById()` already knows you are searching by ID.

You pass only the ID name:

```
mainTitle
```

not:

```
#mainTitle
```

29. What Happens If the ID Exists?

HTML:

```
<h1 id="mainTitle">Recipe</h1>
```

JavaScript:

```
const title = document.getElementById("mainTitle");  
console.log(title);
```

Conceptual output:

```
<h1 id="mainTitle">Recipe</h1>
```

The method returns the matching element.

30. What Happens If the ID Is Wrong?

HTML:

```
<button id="runBtn">Run</button>
```

JavaScript:

```
const button = document.getElementById("wrongId");  
console.log(button);
```

Expected output:

null

Simple meaning:

null means JavaScript searched but found nothing.

31. Why null Can Cause Errors

If an element search returns null, you cannot use element properties or methods on it.

Problem example:

```
const button = document.getElementById("wrongId");

button.addEventListener("click", () => {
  console.log("Clicked");
});
```

This fails because button is null.

The problem is similar to trying this:

```
null.addEventListener("click", () => {});
```

That does not work because null is not an element.

Safer version:

```
const button = document.getElementById("wrongId");

if (button !== null) {
  button.addEventListener("click", () => {
    console.log("Clicked");
  });
}
```

Beginner rule:

If an element might not exist, check for null before using it.

Part I: `querySelector()`

32. What Is `querySelector()`?

`querySelector()` finds the first element that matches a CSS selector.

Syntax:

```
document.querySelector("selector");
```

Example HTML:

```
<h1>Title</h1>
<p class="note">First note</p>
<p class="note">Second note</p>
```

Example JavaScript:

```
const firstNote = document.querySelector(".note");
console.log(firstNote.textContent);
```

Expected output:

```
First note
```

Why only `First note`?

Because `querySelector()` returns only the first matching element.

33. CSS Selectors in `querySelector()`

`querySelector()` uses CSS selector syntax.

Examples:

```
document.querySelector("p");
```

Finds the first `<p>` element.

```
document.querySelector(".note");
```

Finds the first element with `class="note"`.

```
document.querySelector("#mainTitle");
```

Finds the element with `id="mainTitle"`.

```
document.querySelector("ul li");
```

Finds the first `<li>` inside a `<ul>`.

34. Tag Selectors

A tag selector selects by HTML tag name.

HTML:

```
<p>Hello</p>
<p>World</p>
```

JavaScript:

```
const paragraph = document.querySelector("p");
console.log(paragraph.textContent);
```

Expected output:

```
Hello
```

Because `querySelector("p")` finds the first `<p>`.

35. Class Selectors

A class selector starts with a dot:

```
.note
```

HTML:

```
<p class="note">Remember this</p>
```

JavaScript:

```
const note = document.querySelector(".note");  
console.log(note.textContent);
```

Expected output:

```
Remember this
```

Beginner rule:

Use `.` before a class name in `querySelector()`.

36. ID Selectors in `querySelector()`

An ID selector starts with `#`.

HTML:

```
<h1 id="mainTitle">Dashboard</h1>
```

JavaScript:

```
const title = document.querySelector("#mainTitle");  
console.log(title.textContent);
```

Expected output:

```
Dashboard
```

Beginner rule:

Use `#` before an ID name in `querySelector()`.

Important contrast:

```
document.getElementById("mainTitle"); // no #  
document.querySelector("#mainTitle"); // uses #
```

37. Descendant Selectors

A descendant selector selects an element inside another element.

HTML:

```
<ul class="ingredients">  
  <li>Flour</li>  
  <li>Sugar</li>  
</ul>
```

JavaScript:

```
const firstIngredient = document.querySelector("ul.ingredients li");  
console.log(firstIngredient.textContent);
```

Expected output:

```
Flour
```

Simple meaning:

Find the first `<li>` inside `<ul class="ingredients">`.

38. What If `querySelector()` Finds Nothing?

If `querySelector()` finds nothing, it returns `null`.

Example:

```
const missingItem = document.querySelector(".missing");
console.log(missingItem);
```

Expected output:

```
null
```

This can fail:

```
console.log(missingItem.textContent);
```

Why?

Because `missingItem` is `null`, not an element.

Safer version:

```
const missingItem = document.querySelector(".missing");

if (missingItem !== null) {
  console.log(missingItem.textContent);
}
```

Part J: `querySelectorAll()`

39. What Is `querySelectorAll()`?

`querySelectorAll()` finds all elements that match a CSS selector.

Syntax:

```
document.querySelectorAll("selector");
```

Example HTML:

```
<ul class="ingredients">
  <li>Flour</li>
```



```
<li>Sugar</li>
<li>Milk</li>
</ul>
```

Example JavaScript:

```
const items = document.querySelectorAll("ul.ingredients li");

items.forEach(item => {
  console.log(item.textContent);
});
```

Expected output:

```
Flour
Sugar
Milk
```

Simple meaning:

Find every matching element, then loop through them.

40. `querySelector()` vs `querySelectorAll()`

HTML:

```
<p class="note">First note</p>
<p class="note">Second note</p>
<p class="note">Third note</p>
```

Using `querySelector()`:

```
const note = document.querySelector(".note");
console.log(note.textContent);
```

Expected output:

```
First note
```

Using `querySelectorAll()`:

```
const notes = document.querySelectorAll(".note");

notes.forEach(note => {
  console.log(note.textContent);
});
```

Expected output:

```
First note
Second note
Third note
```

Simple comparison:

Method	Finds	Returns
<code>querySelector()</code>	First match only	One element or <code>null</code>
<code>querySelectorAll()</code>	All matches	A NodeList

Part K: NodeList

41. What Is a NodeList?

A NodeList is a collection of DOM nodes.

You usually get a NodeList from `querySelectorAll()`.

Example:

```
const items = document.querySelectorAll("li");
console.log(items);
```

Conceptual output:

```
NodeList(3) [li, li, li]
```

Beginner meaning:

A NodeList is a list-like collection of selected DOM elements.

42. NodeList Is Not Exactly an Array

A NodeList looks like an array in some ways:

```
const items = document.querySelectorAll("li");
console.log(items[0]);
console.log(items.length);
```

You can access items by index.

You can check `.length`.

You can often use `.forEach()`.

But it is not exactly the same as a normal JavaScript array.

For example, array methods like `map()` may not be available directly on every NodeList environment.

Beginner rule:

Treat a NodeList as a list-like DOM collection, not as a full array.

43. Looping Through a NodeList

You can commonly use `forEach()`:

```
const items = document.querySelectorAll("li");

items.forEach(item => {
  console.log(item.textContent);
});
```

You can also use `for...of`:

```
const items = document.querySelectorAll("li");

for (const item of items) {
```

```
    console.log(item.textContent);  
  }  
}
```

Both can print each selected element's text.

44. Converting a NodeList to an Array

If you need a real array, use spread syntax:

```
const items = document.querySelectorAll("li");  
const itemArray = [...items];
```

Now `itemArray` is a true array.

Then you can use array methods like:

```
const texts = itemArray.map(item => item.textContent);  
console.log(texts);
```

Simple meaning:

Spread syntax copies the NodeList items into a new array.

45. Empty NodeList

If `querySelectorAll()` finds no matches, it returns an empty NodeList.

Example:

```
const missingItems = document.querySelectorAll(".missing");  
console.log(missingItems.length);
```

Expected output:

```
0
```

This is safe:

```
missingItems.forEach(item => {  
  console.log(item.textContent);  
});
```

The loop simply runs zero times.

Important distinction:

```
document.querySelector(".missing");    // null  
document.querySelectorAll(".missing"); // empty NodeList
```

Part L: Reading Text with `textContent`

46. What Is `textContent`?

`textContent` reads the text inside an element.

HTML:

```
<h1 id="title">Recipe</h1>
```

JavaScript:

```
const title = document.getElementById("title");  
console.log(title.textContent);
```

Expected output:

```
Recipe
```

Simple meaning:

`textContent` gives you the text stored inside an element.

47. `textContent` with List Items

HTML:

```
<ul>
  <li>Flour</li>
  <li>Sugar</li>
  <li>Milk</li>
</ul>
```

JavaScript:

```
const items = document.querySelectorAll("li");

items.forEach(item => {
  console.log(item.textContent);
});
```

Expected output:

```
Flour
Sugar
Milk
```

Part M: Complete Sprint 11 Example

48. Full HTML Example

Create a file called `index.html`:

```
<!DOCTYPE html>
<html>
  <body>
    <h1 id="mainTitle">Recipe</h1>

    <section class="recipe-card">
      <h2>Ingredients</h2>

      <ul class="ingredients">
        <li>Flour</li>
        <li>Sugar</li>
        <li>Milk</li>
      </ul>
    </section>
```

```
<script>
  const title = document.getElementById("mainTitle");
  const section = document.querySelector("section");
  const ingredients = document.querySelectorAll("ul.ingredients li");

  console.log(title.textContent);
  console.log(section);
  console.log(ingredients);

  ingredients.forEach(ingredient => {
    console.log(ingredient.textContent);
  });
</script>
</body>
</html>
```

49. Expected Console Output

```
Recipe
<section class="recipe-card">...</section>
NodeList(3) [li, li, li]
Flour
Sugar
Milk
```

50. Step-by-Step Explanation

Step 1

```
const title = document.getElementById("mainTitle");
```

Finds this element:

```
<h1 id="mainTitle">Recipe</h1>
```

Stores it in the variable `title`.

Step 2

```
const section = document.querySelector("section");
```

Finds the first `<section>` element on the page.

Stores it in the variable `section`.

Step 3

```
const ingredients = document.querySelectorAll("ul.ingredients li");
```

Finds all `<li>` elements inside:

```
<ul class="ingredients">
```

Stores the matching elements in a NodeList called `ingredients`.

Step 4

```
console.log(title.textContent);
```

Reads the text inside the title element.

Output:

```
Recipe
```

Step 5

```
ingredients.forEach(ingredient => {  
  console.log(ingredient.textContent);  
});
```

Loops through each ingredient element.

Prints each ingredient's text.

Output:

```
Flour
Sugar
Milk
```

Part N: Method Comparison

51. `getElementById()` vs `querySelector()` vs `querySelectorAll()`

Method	Returns	Use When
<code>getElementById("id")</code>	One element or <code>null</code>	You have a unique ID
<code>querySelector("selector")</code>	First matching element or <code>null</code>	You want one element using CSS selector syntax
<code>querySelectorAll("selector")</code>	NodeList, possibly empty	You want all matching elements

52. Return Value Summary

Code	If Found	If Not Found
<code>document.getElementById("title")</code>	Element	<code>null</code>
<code>document.querySelector(".card")</code>	First matching element	<code>null</code>
<code>document.querySelectorAll(".card")</code>	NodeList with items	Empty NodeList

Beginner rule:

One-element searches can return `null`. Many-element searches return a `NodeList`, even when empty.

Part O: Common Beginner Mistakes

53. Mistake 1: Forgetting `.` for Class Selectors

HTML:

```
<p class="note">Remember this</p>
```

Wrong:

```
document.querySelector("note");
```

This searches for a `<note>` tag.

Correct:

```
document.querySelector(".note");
```

This searches for an element with `class="note"`.

Beginner rule:

Class selectors need a dot in `querySelector()` and `querySelectorAll()`.

54. Mistake 2: Forgetting `#` for ID Selectors in `querySelector()`

HTML:

```
<h1 id="mainTitle">Dashboard</h1>
```

Wrong:

```
document.querySelector("mainTitle");
```

This searches for a `<mainTitle>` tag.

Correct:

```
document.querySelector("#mainTitle");
```

This searches for the element with `id="mainTitle"`.

Beginner rule:

ID selectors need `#` when using `querySelector()`.

55. Mistake 3: Using `#` with `getElementById()`

Wrong:

```
document.getElementById("#mainTitle");
```

Correct:

```
document.getElementById("mainTitle");
```

Why?

`getElementById()` already knows it is searching for an ID.

56. Mistake 4: Treating One Element Like Many Elements

Wrong:

```
const title = document.querySelector("h1");

title.forEach(item => {
  console.log(item.textContent);
});
```

Why wrong?

`querySelector()` returns one element, not a collection.

Correct:

```
const titles = document.querySelectorAll("h1");

titles.forEach(title => {
  console.log(title.textContent);
});
```

Use `querySelectorAll()` when you want a collection.

57. Mistake 5: Treating Many Elements Like One Element

Wrong:

```
const items = document.querySelectorAll("li");
console.log(items.textContent);
```

Why wrong?

`items` is a `NodeList`, not one element.

A `NodeList` does not have one single `textContent` value.

Correct:

```
const items = document.querySelectorAll("li");

items.forEach(item => {
  console.log(item.textContent);
});
```

58. Mistake 6: Using an Element Before Checking for `null`

Problem:

```
const message = document.querySelector(".message");
console.log(message.textContent);
```

If no `.message` element exists, `message` is `null`.

Then this fails:

```
message.textContent
```

Safer version:

```
const message = document.querySelector(".message");

if (message !== null) {
  console.log(message.textContent);
}
```

59. Mistake 7: Running JavaScript Before HTML Exists

This can fail:

```
<script>
  const button = document.getElementById("runBtn");
  console.log(button);
</script>

<button id="runBtn">Run</button>
```

Why?

The script runs before the browser reaches the button.

At the time JavaScript runs, this button does not exist yet:

```
<button id="runBtn">Run</button>
```

So JavaScript may return:

```
null
```

60. Simple Fix for Script Placement

Place the `<script>` near the end of the `<body>`:

```
<button id="runBtn">Run</button>

<script>
  const button = document.getElementById("runBtn");
  console.log(button);
</script>
```

Now the browser creates the button first.

Then JavaScript searches for it.

Beginner rule:

Put your script near the end of `<body>` so the HTML exists before JavaScript tries to select it.

Part P: Practice Example

61. Practice HTML

Use this HTML:

```
<!DOCTYPE html>
<html>
  <body>
    <h1 id="pageTitle">Student Dashboard</h1>

    <section class="student-card" data-level="beginner">
      <h2>Sangeeth</h2>
      <p class="status">Active</p>
      <ul class="skills">
        <li>HTML</li>
        <li>CSS</li>
        <li>JavaScript</li>
      </ul>
    </section>

    <script>
      // Write your Sprint 11 code here
    </script>
  </body>
</html>
```

62. Practice Task Code

Write this inside the `<script>` tag:

```
const pageTitle = document.getElementById("pageTitle");
console.log(pageTitle.textContent);

const card = document.querySelector(".student-card");
console.log(card);

const status = document.querySelector(".status");
console.log(status.textContent);

const skills = document.querySelectorAll(".skills li");

skills.forEach(skill => {
  console.log(skill.textContent);
});
```

63. Expected Output

```
Student Dashboard
<section class="student-card" data-level="beginner">...</section>
Active
HTML
CSS
JavaScript
```

64. Practice Explanation

This line:

```
const pageTitle = document.getElementById("pageTitle");
```

Finds:

```
<h1 id="pageTitle">Student Dashboard</h1>
```

This line:

```
const card = document.querySelector(".student-card");
```

Finds the first element with class `student-card`.

This line:

```
const status = document.querySelector(".status");
```

Finds:

```
<p class="status">Active</p>
```

This line:

```
const skills = document.querySelectorAll(".skills li");
```

Finds every `<li>` inside the element with class `skills`.

That means it finds:

```
<li>HTML</li>
<li>CSS</li>
<li>JavaScript</li>
```

This loop:

```
skills.forEach(skill => {
  console.log(skill.textContent);
});
```

Prints the text from each selected skill.

Part Q: Mini Drills

65. Drill 1: Select by ID

HTML:

```
<h1 id="appTitle">My App</h1>
```

Task:

Select it and print its text.

Answer:

```
const appTitle = document.getElementById("appTitle");  
console.log(appTitle.textContent);
```

Expected output:

```
My App
```

66. Drill 2: Select First Class Match

HTML:

```
<p class="message">First</p>  
<p class="message">Second</p>
```

Task:

Select the first message and print its text.

Answer:

```
const message = document.querySelector(".message");  
console.log(message.textContent);
```

Expected output:

First

67. Drill 3: Select All Class Matches

HTML:

```
<p class="message">First</p>
<p class="message">Second</p>
```

Task:

Print every message.

Answer:

```
const messages = document.querySelectorAll(".message");

messages.forEach(message => {
  console.log(message.textContent);
});
```

Expected output:

First
Second

68. Drill 4: Check for `null`

Task:

Safely select an element with class `warning` and print its text only if it exists.

Answer:

```
const warning = document.querySelector(".warning");

if (warning !== null) {
```

```
console.log(warning.textContent);  
}
```

Part R: Sprint 11 Mental Models

69. Main Mental Model

```
HTML file  
  ↓ browser reads it  
DOM tree  
  ↓ JavaScript accesses it  
selected elements  
  ↓ JavaScript reads or uses them  
console output / later page updates
```

Simple meaning:

HTML becomes the DOM. JavaScript uses `document` to access the DOM.

70. Selector Mental Model

```
document.querySelector(".note")
```

Read it as:

Search the document for the first element matching the CSS selector `.note`.

```
document.querySelectorAll(".note")
```

Read it as:

Search the document for all elements matching the CSS selector `.note`.

```
document.getElementById("mainTitle")
```

Read it as:

Search the document for the element whose ID is exactly `mainTitle` .

Part S: Common Patterns

71. Pattern 1: Select One Element by ID

```
const element = document.getElementById("someId");
```

Use when:

- there is one specific element
 - it has a unique ID
-

72. Pattern 2: Select One Element by CSS Selector

```
const element = document.querySelector(".someClass");
```

Use when:

- you want the first matching element
 - you want to use CSS selector syntax
-

73. Pattern 3: Select Many Elements

```
const elements = document.querySelectorAll(".someClass");
```

Use when:

- there may be multiple matching elements
 - you want a NodeList
 - you plan to loop through them
-

74. Pattern 4: Loop Through Selected Elements

```
const items = document.querySelectorAll("li");
```

```
items.forEach(item => {  
  console.log(item.textContent);  
});
```

Use when:

- you selected multiple elements
- you want to do something with each one

75. Pattern 5: Safe One-Element Selection

```
const element = document.querySelector(".message");  
  
if (element !== null) {  
  console.log(element.textContent);  
}
```

Use when:

- the element may or may not exist
- you want to avoid errors

Part T: Sprint 11 Checkpoint Questions

Answer these without looking back.

1. What does DOM stand for?
2. What is the DOM in simple words?
3. Why does JavaScript need the DOM?
4. What is a node?
5. What is an element?
6. What is the difference between a node and an element?
7. What is a parent element?
8. What is a child element?
9. What is a sibling element?
10. What is an ancestor element?
11. What is a descendant element?
12. What does `document` represent?
13. What does `window` represent?
14. What does `navigator` represent?
15. What does `document.getElementById("title")` do?
16. Does `getElementById()` use `#`?

17. What does `getElementById()` return if the element exists?
 18. What does `getElementById()` return if the element does not exist?
 19. What does `querySelector(".card")` return?
 20. What does `querySelectorAll(".card")` return?
 21. What is a `NodeList`?
 22. Is a `NodeList` exactly the same as an array?
 23. How can you loop through a `NodeList`?
 24. How can you convert a `NodeList` into an array?
 25. Why does `.note` mean class in a selector?
 26. Why does `#note` mean ID in a selector?
 27. What is the difference between `querySelector()` and `querySelectorAll()`?
 28. Why can `querySelectorAll()` safely return an empty collection?
 29. Why can `querySelector()` cause an error if you immediately use `.textContent`?
 30. Why is script placement important in beginner DOM code?
-

Part U: Minimum Passing Standard

You are ready to move to Sprint 12 when you can explain every line of this code:

```
const container = document.getElementById("container");
const firstSection = document.querySelector("section");
const items = document.querySelectorAll("ul.ingredients li");

items.forEach(item => {
  console.log(item.textContent);
});
```

You should be able to say:

```
const container = document.getElementById("container");
```

This finds one element with `id="container"`.

```
const firstSection = document.querySelector("section");
```

This finds the first `<section>` element.

```
const items = document.querySelectorAll("ul.ingredients li");
```

This finds all `<li>` elements inside `<ul class="ingredients">`.

```
items.forEach(item => {  
  console.log(item.textContent);  
});
```

This loops through every selected item and prints its text.

Part V: Final Sprint 11 Summary

Sprint 11 teaches the first major browser-control skill in JavaScript.

The core ideas are:

1. The browser turns HTML into the DOM.
2. The DOM is a tree of nodes.
3. HTML elements are nodes JavaScript can select.
4. `document` lets JavaScript access the webpage.
5. `window` represents the browser window.
6. `navigator` gives browser/device information.
7. `getElementById()` finds one element by ID.
8. `querySelector()` finds the first matching element using a CSS selector.
9. `querySelectorAll()` finds all matching elements and returns a `NodeList`.
10. One-element searches may return `null`.
11. Many-element searches return a `NodeList`, even if empty.
12. `textContent` reads text from an element.
13. Selector syntax matters: `.` for class, `#` for ID.
14. Script placement matters because JavaScript can only select elements that already exist in the DOM.

Final beginner sentence:

Sprint 11 is about learning how JavaScript finds and reads parts of a webpage through browser APIs, especially the DOM API.